SmartBIS 운영 수집 설계 (GPS 없음)

> 기준일: 2026-11-03. 이 문서는 Codex가 구현 기준으로 읽는다. 변경 이력은 없으며, 현재 기준의 최종 내용만 적는다.

0. 표기·용어

0.1 표기

- **[확인]**: 업로드 자료 또는 API 문서에서 직접 확인한 사실.
- **[결정]**: 사용자가 결정한 사항. 구현 기준이다.
- **[자료없음]**: 판단에 필요한 원자료가 없어 확인할 수 없는 사항. 원자료가 확보될 때까지 해당 기능은 대상에서 뺀다.
- **[데이터검증]**: 자료로는 판단할 수 없고 수집 데이터로만 확인할 수 있는 사항. 관측 기간에 확인한다.
- **[추정]**: 근거가 부족한 가정. 실측 전에는 사실로 쓰지 않는다. placeholder 값에 붙인다.
- **[계산]**: 확인된 숫자로 계산한 값. 계산식을 함께 적는다.

0.2 프로세스 번호

- ① 위치 수집
- ③ 판정(전날 밤 실행, 스윕·창 관측 결과 해석)
- ⑤ 앱 API
- ⑥ 화면
- 이 문서는 ①·③·⑤·⑥만 다룬다. ②·④는 이 문서 범위 밖이다.

0.3 용어

- **route_id**: TAGO 노선 ID. 노선 번호가 아니다. 전체 105개.
- **node_order(순번)**: route_id 안에서 기점을 0번으로 해 끝까지 1씩 증가하는 번호. 상·하행이 바뀌어도 0으로 돌아가지 않는다.
- **node_id**: 정류장 고유 ID. 이름이 같아도 node_id가 다르면 맞은편 정류장이다.
- **div**: 차량의 운행지역 코드. ud/ld = 관내 북/남, js/gj = 100번 장성발/광주발, hjs/hgj = 휴일 100번, rb = 예비.
- **패턴**: div 안에서 한 대가 하루에 맡는 운행 묶음(예: ud1). 운행패턴.csv 한 행이 한 패턴이다.
- **회차**: 패턴 안의 한 번 운행. 회차마다 기준 출발 시각과 route_id가 있다.
- **기준 출발 시각**: 운행패턴.csv의 회차 시각. 기점 출발 시각으로 해석한다. **[데이터검증]**
- **창(window)**: lane B가 호출하는 시간 구간. 회차 1개당 창 1개다.
- **방문**: 차량이 한 정류장에 접근하는 한 번의 과정.
- **보간**: 중간 정류장 관측이 빠졌을 때 통과로 채우는 처리. 상태 추적과 학습에만 쓴다.
- **placeholder**: 실제 값이 없을 때 쓰는 임시값. 실측으로 취급하지 않는다.

0.4 숫자의 뜻

값	뜻	근거
05:15~21:15	위치 호출 허용 시간대. 첫 운행(06:15, 100번 05:30) 전 여유, 마지막 운행(21:00) 후 15분 여유	시간대 정책

값	뜻	근거
21:15~05:15	위치 호출 금지(코드 고정)	시간대 정책
9,500	하루 위치 호출 상한. 한도 10,000에서 여유 500을 뺀 값	budget.daily_limit
재시도 2회, 10초	실패한 호출을 최대 2번 다시 하며, 재시도 사이 10초 쉰다	budget.retry_max, retry_gap_sec
10분	lane A·B 호출 주기	정책 키
06:00~21:00	lane C(백업 스윕) 시간대	정책 키
30분	lane C 1회전 주기	location.laneC.cycle_min
105	lane C 대상 route_id 수(전체)	10장
2초	lane C에서 ID 사이 간격	location.laneC.gap_sec
3,150	lane C 1일 호출 = 105 × 30회	6장
3분	도착 조회 기준. 예상 소요가 3분 미만이면 방문당 최대 2회 조회	5.3
1.2분	정류장 간 소요 placeholder	[추정]
60분	창 시작 후 강제 종료 시간	4.3, placeholder
5분	마지막 순번 관측 후 창 종료까지 시간	4.3

1. 전제

- 차량 단말에 GPS가 없다. 차량 위치는 TAGO 위치 API(`getRouteAcctoBusLcList`)로만 알 수 있다. **[확인]**
- 차량 상태는 수집 데이터로만 판단한다. **[확인]**
- 위치 API는 노선 단위다. 1회 호출로 그 노선의 모든 차량(차량번호·위치)이 온다. **[확인]**
- 도착정보 API 응답에는 차량번호가 없다. 같은 노선·방향에 차량이 여러 대면 구분이 어렵다. **[확인]**
- TAGO 갱신 주기: 노선·정류장 정보 1회/일, 도착·위치 정보 10~20초. **[확인]**(API 문서)
- 한도: API별 하루 10,000건(개발계정 기준). 운영 예산은 위치 호출 9,500건이다. 운영 계정에 같은 한도가 적용되는지는 **[자료없음]**이며, 9,500 가드로 흡수한다.
- 운행 시간: 관내 06:15~21:00, 100번 05:30~. **[확인]**(사용자 제공)
- 운행 횟수는 6장의 계산 기준(운행패턴.csv 회차 합계)을 쓴다. **[확인]**
- 관측 시작: 화요일 06:00(웹 테스트 환경). 조건은 13.2의 관측 전 필수 단계 완료다. **[결정]**

2. 자료

2.1 파일

- 기본 폴더: `D:\PROJECT\SmartBIS_jangsung\노선 및 운행 관련 자료\군민운수 운행관리시스템\`
- 사용 파일 4개(최종 원자료). 다른 파일명은 쓰지 않는다. **[확인]**
  - `차량구분.csv`
  - `운행패턴.csv`
  - `배차.csv`
  - `장성군 노선 정보_261021_ts.csv`

2.2 차량구분.csv (차량 → 운행지역)

- 열: `vehicle_no`(차량번호), `div`(운행지역), `description`(영문 설명).
- 역할: 차량이 어느 지역에 배정되는지. 지역 안에서 무엇을 운행하는지는 담지 않는다.
- 구성 **[확인]**(33대, 중복 없음):
  - js 10대: 7502, 7505, 7511, 7520, 7521, 7523, 7527, 7528, 7530, 7536
  - gj 2대: 7501, 7513
  - ld 8대: 7500, 7512, 7515, 7517, 7519, 7522, 7526, 7535
  - ud 10대: 7506, 7507, 7508, 7509, 7518, 7532, 7525, 7529, 7531, 7533
  - rb 3대(예비, 운행지역 없음): 7503, 7510, 7516
- 7523은 js, 7532는 ud다. 원본 CSV에 반영되어 있다. **[확인]**
- 7536: 2026-10-31부터 장성터미널↔광주첨단제일풍경채(첨단100)에 투입된다. 운행패턴.csv에 js10 행이 없고 TAGO route_id도 없다. 따라서 **수집 대상에서 제외한다.** route_id 확보 후 별도 요청으로 추가한다. **[자료없음]**

2.3 운행패턴.csv (지역 안의 운행 묶음)

- 역할: 지역 안에서 무엇을 어떤 순서로 운행하는지. 누가 맡는지는 담지 않는다.
- 구조 **[확인]**: 한 행 = 한 패턴(div + 패턴명). 이후 열은 회차마다 (순번, 시각, route_id) 세 칸씩 반복된다.
- 패턴 39개 **[확인]**: ud 10, ld 8, js 9, gj 2, hjs 8, hgj 2.
- 회차 합계 **[확인]** 426회:
  - 관내(ud·ld) 208회
  - 평일 100번(js·gj) 114회
  - 휴일 100번(hjs·hgj) 104회
- 패턴별 회차 예: ud1 14회, ud3 18회, ld2 7회.
- 노선 ID 접두어 **[확인]**: 파일 안의 모든 ID가 `TSB342...`다. 정규화 대상은 0건이다. 그래도 코드에는 `TBS342...` →`TSB342...` 정규화를 둔다(방어용).
- 한 행 = 차량 1대의 하루 운행이라는 해석은 **[데이터검증]**이다. 관측으로 확인한다.
- 시각은 기준 출발 시각이다. **[데이터검증]**
- js10 패턴(7536용)은 없다. **[확인]**

2.4 배차.csv (배정 규칙의 예시)

- 역할: 언제 누가 무엇을 맡는지의 **규칙 예시**다. 판정 입력으로 쓰지 않는다. **[결정]**
- 규칙 **[확인]**:
  - 관내(ud·ld): 매주 패턴이 바뀐다. 예: 7506이 이번 주 ud1이면 다음 주는 ud2.
  - 100번(js·gj): 매일 패턴이 바뀐다.
  - 휴일 100번(hjs·hgj): 휴일 안에서 매일 바뀐다.
- 차량이 어느 패턴을 맡는지는 수집 데이터로 추정한다. **[결정]**

2.5 차량 변경 관리

- 차량과 노선이 계속 맞지 않으면 "변경 후보"로 기록한다.
- **1주 연속 불일치**이면 대시보드에 "관리자 확인 대기"로 표시한다. **[결정]**
- 관리자가 확인하기 전에는 배정 정보를 자동으로 바꾸지 않는다. **[결정]**
- 차량 변경은 수리 등 기간 한정이라고 예상한다. **[추정]**

3. 차량 상태와 정류장 관측 보간

상태	뜻	호출
SEARCH	단말은 켜졌으나 노선을 모름	lane C 스윕 대상
TRACK	노선을 알고 있음(순번을 알면 이후 운행 예측)	운행 중인 노선을 주기적으로 조회
LAYOVER	운행을 마치고 대기 중	다음 출발 2분 전까지 호출 없음. 순번을 모르면 직전 노선만 감시(최대 30분, 설계값)

- **통과 판정**: 응답에는 정류장 통과 시각이 없다. `observed_at`은 저장 시각이다. 통과는 `node_order` 증가로만 판정한다. **[확인]**
- **보간**: 같은 route_id 안에서 연속된 두 관측의 `node_order`가 증가하고, 사이 누락이 5개 이하이며 30분 이하이면 통과로 보고 추정 시각을 넣는다(`interpolated=true`).
  - 누락 6개 이상 또는 30분 초과: "구간 미확인".
  - 순번이 감소하거나 route_id가 바뀌면 보간하지 않는다. **[결정]**
- 보간 행은 상태 추적과 학습에만 쓴다. 운행 판정(일치·불일치·미관측)에는 실제 관측만 쓴다. **[결정]**

4. 위치 API 호출 (①)

4.1 공통

- **DB 적용**: backend는 Flyway를 쓰지 않는다. `server/database/migrations`를 V001→V019 순서로 `psql`에 적용한다. **[확인]**
- **적용 범위**: ①만 TAGO 위치 API를 호출한다. ⑤·⑥은 TAGO를 호출하지 않고 DB만 읽는다. **[결정]**
- **호출 시간대**: 05:15~21:15에만 호출한다. 21:15~05:15 호출 시도는 로그만 남긴다(코드 고정). **[결정]**
- **예산 가드**: 하루 9,500건. 재시도 최대 2회, 재시도 간격 10초. 실패한 시도도 예산에 포함한다. 사용량은 `guard_usage_daily`에 기록하고 재시작 시 당일 값을 복원한다. 상한에 도달하면 그날 위치 호출을 멈추고 로그를 남긴다. **[결정]**
- **lane 분리**: 세 lane은 각자 스레드·간격·throttle을 가진다. 도착정보 호출과 throttle을 공유하지 않는다. **[결정]** 현재 코드의 분리 여부는 **[데이터검증]**(코드 확인 필요).
- 관측 모드도 이 호출 규칙을 그대로 쓴다(8장).

4.2 Lane A (장성100 상시)

- **대상 3개**: TSB342000102, TSB342000103, TSB342000155. 근거: `장성군 노선 정보_261021_ts.csv`의 ROUTE_NM이 `장성100`인 3행. **[확인] [결정]**
- 호출: 05:15~21:15 동안 10분 주기. ID당 96회/일, 합계 288회/일. **[계산]**
- 이 3개는 lane B에서 제외한다.

4.3 Lane B (회차 창)

- **대상 100개 ID**: 관내 86개(운행패턴 ud·id에 있는 ID) + 변형 100번 14개. **[결정]**
  - 변형 100번 14개: TSB342000104, 105, 106, 107, 108, 109, 110, 111, 112, 117, 118, 119, 153, 154. 이름 전부 노선 정보 파일에서 확인. **[확인]**
  - 제외: 장성100 3개(lane A), 006·074(시간표 없음, lane C만).
- **창 시작**: 기준 출발 시각 − 15분(관내), − 10분(변형 100번). **[결정]**
  - 예: 118번(월정100) 기준 출발 07:15 → 창 시작 07:05.
  - 예: 153번(삼서100) 기준 출발 06:45 → 창 시작 06:35.

- **창 종료(정상)**: 해당 route_id에서 이 창 안에 **마지막 순번(node_order)을 처음 관측한 시점 + 5분**. **[결정]**
  - 편도 소요 시간으로 창 끝을 계산하지 않는다. **[결정]**
- **창 종료(미관측 강제)**: 마지막 순번을 끝내 관측하지 못하면 **창 시작 + 60분**에 강제 종료한다. **[결정]** 60분은 placeholder이며 관측 후 조정한다. **[추정]**
- 창 안에서만 10분 주기로 호출하고, 창 밖에서는 호출하지 않는다. **[결정]**
- 같은 ID의 창이 겹치면 하나로 합친다. **[결정]**
- 창 계산에는 정제된 데이터만 쓴다(9장).

4.4 Lane C (백업 스윕)

- 대상: 105개 route_id 전체(장성100 3개와 006·074 포함, 7536 제외).
- 06:00~21:00 동안 30분마다 1회전한다. 1회전은 ID 사이 2초 간격의 순차 호출이다(105 × 2초 = 210초, 약 3.5분). 1회전이 끝나면 다음 주기까지 쉰다. **[결정]**
- 1일 호출: 105 × 30회 = 3,150회. **[계산]**

4.5 429 대응

- 429가 나오면 lane C 간격을 2초 → 3초 → 5초로 단계적으로 올리고 로그를 남긴다. 단계값은 설정값이다. **[결정]**
- 1회전 전체가 429 없이 끝나면 한 단계 낮춘다. **[추정]**(구현 기본값)

4.6 시간표 없는 ID와 이름

- 운행패턴.csv에 없는 ID: TSB342000006, TSB342000074. 이 둘은 같은 순환의 상·하행 분리 ID다. 대응 문제가 없다. lane C만 적용한다. **[확인]**
- 노선 정보 파일에 이름이 없는 ID는 없다. 105개 전부 이름이 있다. **[확인]**

4.7 판정 분담

- 스윕과 창의 관측 결과는 ③이 전날 밤에 해석한다.
- ①은 판정 결과인 `route_call_plan`만 읽으며 판정 로직을 알지 못한다. **[결정]**

4.8 관측 모드

- 8장을 따른다. 105개 10초 순환은 폐기한다. **[결정]**

5. 도착정보 API

5.1 목적과 규모

- 정류장에서 "다음 차량이 언제 오는지"를 보여준다. 도착 API는 위치 API와 별도 한도(API별 10,000건)를 쓴다. **[확인]**
- 도착 호출량은 **[미계산]**이다. 관측 기간에는 도착 스위치가 꺼져 있으므로 0이다(5.9).

5.2 조회 시점

1. 운행을 시작할 때
2. 다음 정류장 방문이 시작될 때(예상 소요가 3분 미만인 경우에만)
3. 예정 시각이 지났는데 아직 통과하지 않았을 때

- 세 시점 모두 5.3 상한을 따른다. 예상 소요가 3분 이상이면 시점과 무관하게 호출하지 않는다. **[결정]**

5.3 방문당 상한 (C안, 코드 고정)

- **예상 소요 3분 미만**: 방문당 도착 API **최대 2회**.
- **예상 소요 3분 이상**: 도착 API **호출 0회**. 학습된 정류장 간 소요 시간(시간대별 중앙값)을 표시한다. 학습값이 없으면 `정류장 수 × 1.2분`(placeholder)을 쓴다.
- 이 상한은 코드 고정이며 정책 키가 아니다. **[결정]**
- 현재 `ArrivalVisitStore.java:32`(`query_count < 2`)는 2회 조건과 일치한다. **[확인]** DB CHECK 여부는 **[데이터검증]**(코드·DB 확인).
- 긴 구간의 위치 추적은 lane B·C의 위치 값으로 한다. **[결정]**

5.4 짧은 구간 계산

- 쌓인 위치 기록에서 학습한 정류장 간 소요 시간(시간대별 중앙값)으로 계산한다. 학습에는 9장 정제 조건을 통과한 데이터만 쓴다. **[결정]**
- 학습값이 없으면 "마지막 도착 예정 시각 − 경과 시간"을 보여준다. **[결정]**

5.5 응답 매칭

- 응답의 `arrprevstationcnt`(남은 정류장 수)가 차량의 현재 위치와 맞는 값을 고른다. 실측으로 확인한다. **[데이터검증]**

5.6 관내 혼동

- 관내 노선은 같은 노선·방향에 우리 차량이 1대인 경우가 많아 혼동이 적다. **[추정]** 관측 기간에 확인한다.

5.7 2분 위치 기준 폐기

- `ArrivalService.java:96,112`의 "최근 2분 이내 위치 행" 조건은 폐기한다. 설계 근거가 없다. **[결정]**
- 대체 조건은 없다. 도착 호출은 5.2의 세 시점과 5.3의 방문당 상한만 따른다. **[결정]**
- 위치 스위치가 꺼져 있으면 도착이 0건이 되는 것은 코드 경로상의 예상이다. **[추정]**

5.8 보간 행과 수집률

- 보간 행은 학습에만 쓰고 운행 판정에는 쓰지 않는다. **[결정]**
- 수집률(관측 정류장 수 ÷ 노선 정류장 수)은 요약 지표로만 쓴다. 노선 제외 기준이 아니다. **[결정]**

5.9 도착 스위치

- `TAGO_ARRIVAL_ENABLED`는 현재 `false`다. **[확인]**
- 관측 기간에는 `false`로 둔다(위치만 수집). 운영 재개 시 위치 스위치와 함께 `true`로 켠다. **[결정]**

6. 예산 계산

lane	계산	평일 일 호출	휴일 일 호출	구분
C	105 × 30회	3,150	3,150	[계산]
A	3 × 96회	288	288	[계산]

lane	계산	평일 일 호출	휴일 일 호출	구분
B	창 수 상한 × 창당 최대 7회	최대 2,254	최대 728	[계산], 창 길이 [추정]
합계(재시도 제외)		최대 5,692	최대 4,166	9,500 이내

- **창 수 상한**: 평일 322회(관내 208 + 100번 114). 휴일 104회(hjs·hgj). 장성100 3개의 회차는 빼야 하지만 빼지 않은 상한으로 계산했다. **[계산]**
- **창당 최대 7회**: 창 최대 길이 60분을 10분 주기로 호출(0·10·20·30·40·50·60분). **[계산]**, 창 길이는 **[추정]**이다.
- 재시도와 429 여유: 실패 시도도 예산에 포함한다. 최악의 경우 가드가 9,500건에서 호출을 멈춘다. **[결정]**
- 편도 소요 학습값에는 의존하지 않는다. 창 수와 창 길이 상한으로만 계산한다. **[계산]**
- 10,000건 한도의 운영 계정 적용은 **[자료없음]**이다. 9,500 가드로 흡수한다.
- 도착 API 호출량은 **[미계산]**이며 위치 예산과 별도다. 관측 기간에는 0이다.
- 합계 9,500 이하 확인: **충족**(위 표).

7. 한도를 넘을 때의 선택지 (효과가 큰 순)

0. **lane C 주기 연장**: 30분 → 더 긴 주기. `location.laneC.cycle_min`을 먼저 조정한다. **[결정]**
1. **정밀도 낮추기**: 2정류장마다 확인하고 중간은 학습된 소요 시간으로 추정한다. 호출은 약 절반이 되지만 정확도가 떨어진다. **[결정]**
2. **도착정보로 통과 판정**: 같은 노선·방향에 1대만 운행할 때만 쓴다. 관내 노선 대부분이 이 조건에 가깝다. 노선별 일 회차 분포는 운행패턴 기준으로 계산할 수 있다. **[데이터검증]**

8. 관측 모드

- **기간**: 화요일 06:00 ~ 일요일 21:00(평일 4일 + 토·일). **[결정]**
- **장소**: web2 테스트 환경에서 수집한다. 운영 서버는 건드리지 않는다. 결과 확인 후 운영 이관을 별도 승인한다. **[결정]**
- **시작 조건**: 13.2의 관측 전 필수 단계(1~7)와 T1~T15 실패 0. **[결정]**
- **호출 규칙**: 4장의 lane A·B·C를 그대로 쓴다. 관측과 운영의 호출 규칙이 같아야 결과를 운영에 쓸 수 있다. **[결정]**
- **저장**: `observation.mode=true`이면 응답에 나온 **모든 차량**(차량번호, route_id, node_order, 시각)을 저장한다. 저장 동작은 Codex가 확인한다. **[데이터검증]**
- **얻는 값**:
  - 순번 운행표와 실제 운행의 차이(지연·결행·순번 교체)
  - 실제 차량 ↔ route_id 대응(운행패턴 기준과 비교)
  - 창 밖에서 발견된 차량 건수(창 규칙 검증)
  - 기준 출발 시각과 실제 첫 관측 시각의 차이(창 시작점 검증)
  - 정류장 간 실제 소요 시간(시간대별, 정제 후)
  - 같은 노선·방향을 동시에 달리는 비율
  - 토·일에 hjs·hgj 운행표가 적용되는지 여부
  - 미관측 강제 종료 60분의 적정성

9. 기존 코드와의 관계

항목	현재	목표
순번 0 허용	완료. 로직 `18cf33f`, Catalog `8835d90`(로컬 커밋, push 안 함)	유지

항목	현재	목표
위치 API 기점 순번	기점 0 허용 코드 있음	응답의 기점 순번 확인 **[데이터검증]**
lane 창 방식	`18cf33f`. 창 종료는 편도 소요 방식	마지막 순번 관측 + 5분(4.3). 코드·테스트 수정
도착 방문당 상한	2회(`ArrivalVisitStore.java:32`)	C안과 일치. 긴 구간 0회 로직 확인
2분 위치 기준	코드에 존재 (`ArrivalService.java:96,112`)	폐기(5.7)
방향 조회	UP/DOWN 조건 조회. `updowncd=2` 노선 404 가능성	방향 조건 제거, `route_id` + `node_order` + `1`로 다음 정류장 찾기. 404 재현 **[데이터검증]**
노선 ID	TBS 접두어 혼재 가능	TSB 정규화(T12). 파일은 전부 TSB
throttle	lane별 분리 여부 **[데이터검증]**	lane별 분리
TagoQuotaGuard	예산 가드 유지	lane마다 `window`, `min-gap-ms` 설정
정책 조회	미구현	11장. V020
외부 공개 주소	하드코딩 여부 **[데이터검증]**	`PUBLIC_BASE_URL` 설정값

- 테스트 단계(단말 3대, 10초 간격)와 운영의 차이는 설정값과 상태 판단으로만 둔다. 테스트 전용 분기 코드는 만들지 않는다. **[결정]**
- 편도 소요 학습값은 정제 조건을 통과한 데이터만 쓴다. 전환 중앙값 49.76초(p90 약 2,164초)는 날짜·회차 미분리, 순번 감소 774건, 건너뜀 2,551건이 섞인 집계이므로 인접 정류장 통행 시간으로 쓰지 않는다. **[결정]**
- **외부 공개 주소**: 지자체 요구 주소 `jangsung:8443`은 코드에 하드코딩하지 않고 설정값 `PUBLIC_BASE_URL`로 둔다. CORS·리다이렉트·쿠키 도메인 영향은 Codex가 찾아 보고한다. **[결정]**. 점(.)이 없는 `jangsung`은 DNS 조회가 안 될 수 있으므로 정식 도메인명이 필요하다. **[자료없음]**

10. 노선 자료 (TAGO 105개)

10.1 구성 **[확인]**

구분	개수	route_id	근거
관내 (운행패턴 ud·ld에 있음)	86	ud·ld 패턴의 TSB ID	운행패턴.csv
관내 (패턴 없음)	2	TSB342000006, TSB342000074	같은 순환의 상·하행 분리 ID
장성100	3	TSB342000102(0100), 103(0101), 155(0111)	노선 정보 ROUTE_NM `장성100`
변형 100번	14	104, 105, 106, 107, 108, 109, 110, 111, 112, 117, 118, 119, 153, 154	노선 정보 ROUTE_NM

- 합계 105 = 관내 88 + 100번 계열 17. 관내 88 = 패턴 86 + 패턴 없음 2. 100번 계열 17 = 장성100 3 + 변형 14.
- 노선번호 기준: 관내 48개, 100번 계열 이름 8개(장성100, 마령100, 녹진100, 진원100, 삼서100, 비아100, 월정100, 월곡100). **[확인]**
- 변형 100번 이름: 마령100(104·105·106), 녹진100(107·108·109), 진원100(110·111·112), 비아100(117), 월정100(118), 월곡100(119), 삼서100(153·154). **[확인]**
- 구조: 왕복형 74개, 편도형 31개. **[확인]**(노선 정보 방향 값으로 **[데이터검증]**)
- 순번은 route_id 안에서 0부터 끝까지 이어진다. 상·하행이 바뀌어도 다시 시작하지 않는다. **[결정]**
- 7536(첨단100): 카탈로그에 없어 수집 대상이 아니다. route_id 확보 후 추가한다. **[자료없음]**
- 운수사 원자료 64개 노선·333회는 업로드 자료에 없다. 이 설계의 판정은 운행패턴.csv의 TAGO ID를 쓰므로 운수사 원자료와의 대응표는 필요 없다. **[결정]**

11. 설정 분리

11.1 원칙

- **배포 설정(`.env`, `compose.yaml`)**: 서버마다 다른 값과 비밀값만 둔다. DB 접속, 포트, 인증서 경로, 로그 경로, 타임존, `TAGO_SERVICE_KEY`, `PUBLIC_BASE_URL`. **[결정]**
- **기능 스위치(비상 차단)**: `TAGO_ENABLED`, `TAGO_VEHICLE_LOCATION_ENABLED`, `TAGO_ARRIVAL_ENABLED`. 기본값은 `false`다. 스위치가 `false`이면 정책 값과 무관하게 호출하지 않는다. **[결정]**
- **운영 정책(정책 테이블)**: 호출 시간대, lane별 주기, 스윕 주기, 429 백오프 단계, 창 종료 분, 재시도 횟수·간격, 관측 모드, 카탈로그 동기화 주기. 재기동 없이 바꾼다. **[결정]**
- **노선 자료(시간표 테이블)**: 회차별 기준 출발 시각, 편도 소요 학습값, lane A·B 대상 ID, 노선 구조. 운수사 자료 등록으로만 바꾼다. **[결정]**
- **코드 고정(정책 아님)**: 도착 방문당 상한(3분 미만 2회, 3분 이상 0회), 21:15~05:15 위치 호출 금지, 순번 0 기준, 상태 전이 규칙, 일일 예산 하드 상한 검사. **[결정]**

11.2 V020(정책 테이블) 설명

V020이란: DB 스키마 변경 파일 번호다. 지금 적용된 마지막 번호는 V019이고, V020은 그 다음에 붙는 변경이다. 적용은 `psql`로 손으로 실행한다(Flyway 미사용). **[확인]**

V020이 만드는 것:

- `tago_policy` 테이블: 정책 값 하나가 한 행이다. 칼럼은 `policy_key`(키 이름), `value`(값), `value_type`(숫자·시각·목록 등 자료형), `scope_type`(GLOBAL / SERVICE / ROUTE), `scope_id`(적용 대상), `profile_name`(프로필), `updated_by`(변경자), `updated_at`(변경 시각), `note`(설명)다.
- `tago_policy_history` 테이블: 값이 바뀔 때마다 이전 값, 새 값, 변경자, 시각을 남긴다.

V020의 효과: 지금은 호출 주기와 창 분 같은 값이 설정 파일과 코드에 고정되어 있어 바꾸려면 재기동이 필요하다. V020이 적용되고 정책 조회 로직(11.5 단계 1)이 들어가면, 60초마다 DB에서 값을 다시 읽어 재기동 없이 반영한다.

백업과 적용 순서: DB에 테이블을 추가하는 것은 운영 DB에 쓰는 작업이다. 되돌릴 기준점이 있어야 한다.

1. 운영 DB 덤프를 NAS에 받는다: `/homeData/smartbis_backup/db/`, 파일명 `YYYYMMDD_HHMM_<대상>.dump`. **[결정]**
2. 덤프를 임시 DB(`smartbis_restore_test`)에 복원해 덤프가 정상인지 확인한다. 운영 DB에는 복원하지 않는다. **[결정]**
3. V020은 web2 테스트 DB에 먼저 적용해 정책 조회 로직을 검증한다. **[결정]**

4. 관측 기간(화~일)에는 운영 DB에 V020을 적용하지 않는다. 관측은 설정 파일 기본값으로 돈다. **[결정]**
5. 운영 DB 적용은 관측 결과를 본 뒤 별도 승인으로 진행한다. **[결정]**

11.3 정책 키 (초기값)

키	초기값	설명
location.laneA.enabled	true	장성100 상시 수집(대상 3개)
location.laneA.interval_min	10	lane A 주기(분)
location.laneA.window	05:15~21:15	lane A 호출 시간대
location.laneB.lead_min.regular	15	관내 창 시작 = 기준 출발 − 15분
location.laneB.lead_min.variant	10	변형 100번 창 시작 = 기준 출발 − 10분
location.laneB.close_after_last_min	5	마지막 순번 관측 후 창 종료까지 시간(분)
location.laneB.no_observation_max_min	60	마지막 순번 미관측 시 창 시작 후 강제 종료 시간(분)
location.laneB.interval_min	10	lane B 주기(분)
location.laneC.enabled	true	백업 스윕
location.laneC.window	06:00~21:00	스윕 시작 가능 시간대
location.laneC.cycle_min	30	스윕 주기(분). 예산 부족 시 먼저 늘린다
location.laneC.gap_sec	2	ID 간 간격(초). 1~3초 범위
location.laneC.backoff_429_sec	2,3,5	429 시 단계 간격
budget.daily_limit	9500	일일 위치 호출 상한
budget.retry_max	2	재시도 횟수
budget.retry_gap_sec	10	재시도 간격
observation.mode	false	전체 차량 저장 플래그. 운영은 false 고정
catalog.sync_cron	-(차단)	현재 운영 값. 재개 값은 승인 후 결정
arrival.enabled	false	현재 운영 값. 운영 재개 시 위치와 함께 true

- 코드 고정(정책 키 아님): 위치 호출 금지 구간 21:15~05:15, 도착 방문당 상한, 순번 0 기준.

11.4 우선순위와 프로필

- 우선순위: **노선 값 > 서비스 값 > 프로필 기본값 > 코드 안전 기본값**. **[결정]**
- 조회 실패 시 마지막 정상 값을 쓴다. 정상 값이 없으면 위치 수집을 끈 상태로 둔다. **[결정]**
- 정책 값이 코드 고정 규칙을 어기면 저장을 거부하고 로그를 남긴다. **[결정]**
- 프로필 4종(대도시형, 중소도시형, 도농복합형, 농촌형)은 정책 키의 기본값 묶음이다. 장성군은 도농복합형 후보이며, 확정 전까지 11.3 초기값을 쓴다. **[결정]**
- 서비스 구분: 차량 서비스(위치 수집, 창 계산), 정류소 서비스(도착정보), 공공시설 서비스(노선과 무관한 조회). **[결정]**

11.5 단계

- **단계 0(현재)**: 위치 스위치 `false`, 관측 모드 `false`, 카탈로그 동기화 `-`, 도착 스위치 `false`. 운영 backend 재시작 정책 `no`. **[확인]**
- **단계 1(소스 수정, 관측 시작 전 필수)**: 4장 호출 규칙, 창 종료 5분(4.3), 방문당 상한 C안(5.3), 2분 기준 폐기(5.7), TSB 정규화, 관측 모드 저장, `PUBLIC_BASE_URL`. 테스트 T1~T15 실패 0. 실제 TAGO 호출은 하지 않는다. **[결정]**
- **단계 1-b(관측 기간 중 병행)**: 정책 조회(11.2, 11.4). V020은 web2 테스트 DB에서 검증한다. 운영 이관 전에 완료한다. **[결정]**
- **단계 2(운영 안정화 후)**: 관리 API와 화면, 변경 이력 조회, 프로필 선택. **[결정]**

11.6 `.env` 변경 항목

항목	웹 테스트 관측	운영 재개(승인 후)
`TAGO_ENABLED`	true	true
`TAGO_VEHICLE_LOCATION_ENABLED`	true	true
`TAGO_LOCATION_OBSERVATION_MODE`	true	false
`TAGO_ARRIVAL_ENABLED`	false	true
`TAGO_CATALOG_SYNC_CRON`	`-`	`-`(재개 방식 결정 전)

- 운영 `.env`와 운영 `latest` 이미지는 관측 결과 확인 후 별도 승인으로 바꾼다. **[결정]**

12. 검증 기준

12.1 노선 유형별 기점·종점 기대값

- **왕복형(74개)**: 한 route_id 안에서 기점(UP 0번)과 종점(DOWN 마지막 순번)이 같은 정류장 node_id다. 예: TSB342000143, UP 757=0, DOWN 757=76. **[확인]**(이전 자료)
- **편도형(31개)**: 한 ID는 한 방향이고, 순번은 0부터 끝까지 이어진다. 기점과 종점은 서로 다른 정류장이다. **[확인]**
- **공통**: 순번은 route_id 안에서 0부터 끝까지 이어진다. **[결정]**

12.2 연속성 판정 (노선 단위)

- `min(node_order) = 0`
- `count(*) = count(DISTINCT node_order)` (중복 없음)
- `max(node_order) - min(node_order) + 1 = count(*)` (빠짐 없음)
- 세 조건을 모두 만족하면 통과. 판정은 순번만 본다. 샘플 노선(`SAMPLE_ROUTE_001`)은 검사하지 않는다. **[결정]**

12.3 기점 이름 대조

- 기준: 노선도 기점·종점 자료(노선도_기점종점_수정본). 노선별 기점·종점 이름과 DB의 node_id를 비교한다.
- 조인: `bus_route_stops.node_id` = `bus_stops.node_id`. 이름은 `bus_stops.node_name`.
- 판정: 0번 이름과 node_id가 기준 기점과 같으면 통과. `node_name`이 NULL이면 미확인으로 기록한다. **[결정]**
- 1차 대상: 왕복형 대표 ID(0001, 0143), 편도형 대표 ID(0002, 0044, 0105, 0111). **[결정]**

12.4 0번 기준 도착정보 확인

- 대상: 0번 정류장 중 하루 운행이 확인된 노선 2개 이내.
- 방법: 도착 API를 1~2회만 호출한다(방문당 상한 2회). `arrprevstationcnt`가 차량 위치와 맞는지 본다.
- 실제 TAGO 호출은 운영 승인 후 낮 시간에만 한다. 개발 테스트에서는 호출하지 않는다. **[결정]**

12.5 테스트 목록

번호	테스트	통과 기준	현재
T1	창 안·밖 호출	창 안 호출 발생, 창 밖 0건	이전 기준 통과. 창 종료 규칙 변경 후 재작성
T2	21:15~05:15 호출 금지	위치 호출 0건, 로그만	통과(18cf33f)
T3	창 겹침 병합	겹치는 창이 하나로 합쳐짐	이전 기준 통과. 재확인
T4	lane C 1회전	105건, ID 간격 2초	통과(18cf33f)
T5	429 백오프	2→3→5초, 로그 기록	통과(18cf33f)
T6	창 시작 계산	118 기준 07:15 → 07:05, 153 기준 06:45 → 06:35	이전 기준 통과. 재작성
T7	lane 대상	lane A = 102·103·155 3개. lane B에 장성 100 3개 없음	재작성
T8	0번 기준 입력	0143 UP 757 순번 0, DOWN 757 순번 76	통과(`8835d90`)
T9	빌드·테스트	exit 코드, Tests run / Failures / Errors, 실패 0	미통과: tago 103건 중 `RouteCallPlanTest` 1건 실패. 기대값 정정 후 재판정
T10	창 종료	마지막 순번 관측 + 5분에 닫힘, 이후 호출 0건. 미관측 시 창 시작 + 60분에 닫힘	미구현
T11	방문당 C안	예상 3분 미만 도착 호출 최대 2회, 3분 이상 0회, 학습값 없으면 정류장 수 × 1.2분 표시	미구현
T12	TBS→TSB 정규화	`TBS342000033` 입력이 `TSB342000033`으로 매칭	미구현
T13	관측 모드 저장	`observation.mode=true`일 때 응답의 모든 차량 저장, `false`일 때 저장 안 함	미구현
T14	예산 가드	일일 9,500 도달 시 당일 위치 호출 중지 + 로그. 재시도 포함 집계	미구현
T15	호출 대상 집합	lane A 3개, lane B 100개(장성100·006·074 제외), lane C 105개	미구현

- 테스트 실행 기록은 로그 파일로 저장한다. 보고에는 tail 50줄 발췌와 숫자만 쓴다. **[결정]**

13. 작업 범위와 현재 상태

13.1 완료

- lane 창 방식(이전 기준), 21:15~05:15 호출 금지, 429 백오프, 0번 순번 하한: 로컬 커밋 `18cf33f`, `8835d90`. push 안 함. **[확인]**
- 운영 backend 재시작 정책 `no`(compose.yaml 변경, 커밋 전). **[확인]**
- T1~T8 통과(이전 기준). **[확인]**
- 차량구분·운행패턴·노선 정보의 집계 검증(2·10장). **[확인]**

13.2 작업 순서

관측 전 필수 (1~7)

1. T9: `RouteCallPlanTest` 기대값 정정(예외 없음, 상시 조회 생략, 백업 스윕 생성), 실패 0 확인. 기대 정정은 승인됨. **[결정]**
2. 창 종료를 "마지막 순번 관측 + 5분"으로 변경(4.3), 미관측 시 창 시작 + 60분. T1·T3·T6·T7 재작성, T10 추가.
3. 방문당 C안 반영(5.3), T11 추가.
4. 2분 위치 기준 폐기(5.7).
5. TBS→TSB 정규화, T12 추가.
6. 관측 모드 저장(T13), 예산 가드 확인(T14), 호출 대상 집합(T15).
7. `PUBLIC_BASE_URL` 설정과 CORS·리다이렉트 영향 확인.

관측 기간 (8~9) 8. web2 테스트 환경에서 T1~T15 실패 0 확인 후 화요일 06:00 수집 시작. 일요일 21:00 종료. 9. 관측 결과 분석: 6장 예산 실측 재계산, 창 길이 실측으로 60분 placeholder 조정, 편도 소요 학습값 재계산.

관측 후 (10~11) 10. 정책 조회(11.5 단계 1-b), V020을 테스트 DB 검증 후 운영 백업 → 운영 적용 승인 요청. 11. 운영 이관 승인 요청: `.env`, `latest` 이미지 교체(`head-8835d90` 이후 빌드), 위치·도착 재개.

13.3 범위 밖 (실행하지 않음)

- 운영 이미지 교체(`latest`), 운영 `.env` 변경, 운영 DB V020 적용, 위치·도착 재개, push.
- 12장 검증의 실제 TAGO 호출(12.4). 운영 승인 후 낮 시간에만 한다.

13.4 체크리스트

- 로직 수정 범위 확정: ☑
- 0번 순번 하한 수정 및 T8 통과: ☑
- T9 전체 통과(실패 0): ☐
- 창 종료 5분 규칙 + 미관측 60분 반영, T1·T3·T6·T7 재작성, T10: ☐
- 방문당 C안, T11: ☐
- 2분 기준 폐기: ☐
- TBS→TSB 정규화, T12: ☐
- 관측 모드 저장 T13, 예산 가드 T14, 호출 대상 T15: ☐
- `PUBLIC_BASE_URL` 설정: ☐
- 관측 기간 시작(화 06:00): ☐
- 관측 결과 기반 6장 재계산: ☐
- 정책 조회·V020 테스트 DB 검증: ☐
- 운영 DB 백업·V020 적용 승인: ☐
- 운영 이관 승인(`.env`, `latest`, 위치·도착 재개): ☐

14. 결정 사항과 외부 확인

14.1 결정 완료

항목	결정
lane A 대상	TSB342000102, 103, 155
lane B 대상	관내 86 + 변형 100번 14 = 100개
창 종료	마지막 순번 최초 관측 + 5분
미관측 강제 종료	창 시작 + 60분(placeholder)
방문당 도착 상한	C안: 3분 미만 최대 2회, 3분 이상 0회
2분 위치 기준	폐기. 대체 조건 없음
차량 변경 장기화	1주 연속 불일치 → 관리자 확인 대기
배차.csv	판정 입력 아님, 규칙 예시
7536(첨단100)	수집 제외. route_id 확보 후 추가
관측 기간	화요일 06:00 ~ 일요일 21:00
관측 장소	web2 테스트 환경 수집 후 운영 이관
관측 중 도착 스위치	false(위치만 수집). 운영 재개 시 위치와 함께 true
V020	테스트 DB 검증 → 백업 → 운영 적용 별도 승인
RouteCallPlanTest	기대값 정정 승인
장성100 이름·변형 100번 이름	노선 정보 파일로 확인 완료

14.2 외부 확인 (구현과 관측을 막지 않음)

- 10,000건 한도의 운영 계정 적용 여부 **[자료없음]**: 9,500 가드로 흡수.
- 첨단100(7536)의 route_id **[자료없음]**: 확보 시 lane C에 추가.
- 정식 도메인명(`jangsung`에 점이 없음) **[자료없음]**: `PUBLIC_BASE_URL` 설정값으로 대기.
- 토·일의 운행표 적용(hjs·hgj) **[데이터검증]**: 관측에서 확인.
- 운행패턴 한 행 = 차량 1대의 하루 운행 **[데이터검증]**: 관측에서 확인.
- `updowncd=2` 노선의 404 재현 **[데이터검증]**: 관측에서 확인.
- 창 길이 60분과 편도 소요 학습값 **[추정]**: 관측 후 조정.